

神经网络编程框架

南京大学人工智能学院

申富饶，徐百乐

目录

CONTENTS

01. 神经网络框架简介

02. 链式求导与计算图

03. 自动微分

04. 神经网络框架实现

01

神经网络编程框架

神经网络编程框架

- TensorFlow：由Google开发的一个开源机器学习框架，广泛用于各类深度学习任务。它支持多种编程语言，包括Python、C++、Java等。
- PyTorch：由Facebook开发的开源机器学习库，特别适合于科研领域，支持动态图计算。2022年转入Linux Foundation旗下的PyTorch Foundation独立治理。
- JAX：由Google开发，基于XLA编译器，以函数式编程和自动向量化为核心特色，训练效率出众。
- Keras：最初是一个独立的深度学习框架，以其用户友好和易扩展性著称。Keras 2被集成到Tensorflow中作为高级API。Keras 3 转变为后端无关的深度学习库，提供了统一的抽象层屏蔽底层框架差异。
- ONNX：用于表示机器学习模型的开放标准格式，由微软与合作伙伴社区共同创建，旨在提供一套与环境 and 平台无关的通用中间表示格式，解决不同人工智能框架之间的模型互操作性问题。
- PaddlePaddle（百度） / MindSpore（华为） / MegEngine（旷视） **国产框架！**
- Caffe：支持C++和Python语言，适合于图像处理任务。（Caffe2已并入pytorch）
- Theano：由蒙特利尔大学开发的符号计算及深度学习框架，完全基于Python。（已停止维护）
- CNTK：由微软开发，通过有向图描述神经网络，支持多种神经网络模型。（已停止维护）
- MXNet：支持混合符号和命令式编程，具有灵活的编程模型和多语言支持。（已停止维护）

神经网络程序的一般结构

1. 准备训练、验证数据，进行数据预处理；
2. 定义神经网络结构；
3. 迭代训练：
 - 以mini-batch的形式读取数据，送入网络进行前向传播，计算输出值；
 - 根据输出值与学习目标，用损失函数计算出此次迭代的loss；
 - loss进行反向传播，逐层计算梯度、更新参数；
 - 重复此步骤直到达到结束条件，存储模型参数；
4. 读取模型参数和测试数据，将数据送入网络，输出预测值。

神经网络编程框架的功能

- 读取数据与预处理功能;
- 定义张量数据结构, 实现基本运算函数库;
- 模块化的神经网络库, 实现常见的神经网络模型;
- 实现常见的损失函数: 交叉熵, MSE均方损失等;
- 计算反向传播梯度;
- 实现优化器;
- GPU并行计算;
- 储存、读取神经网络。

02

链式求导与计算图

数值微分

函数的导数定义如下：

$$\frac{df(x)}{dx} \Big|_{x=a} = f'(a) = \lim_{x \rightarrow a} \frac{f(x) - f(a)}{x - a}$$

数值计算中，常用差商法计算导数：

$$f'(x) = \frac{f(x+h) - f(x)}{h} + O(h) = \frac{f(x) - f(x-h)}{h} + O(h) = \frac{f(x+h) - f(x-h)}{2h} + O(h^2)$$

这三种近似分别称为数值微分的向前差商、 向后差商和中心差商公式，中心差商公式有更好的精度。

链式求导

神经网络程序中的函数通常是由简单函数复合而来的，因此可以使用“自动微分”技术获得微分的精确表示。这种方法的基础是微分的链式法则：

$$\frac{dg(f(x))}{dx} = \frac{dg}{df} \cdot \frac{df}{dx}$$

以及函数相加与相乘的导数

$$\frac{d(f(x) + g(x))}{dx} = \frac{df(x)}{dx} + \frac{dg(x)}{dx}, \quad \frac{d(f(x) \cdot g(x))}{dx} = g(x) \cdot \frac{df(x)}{dx} + f(x) \cdot \frac{dg(x)}{dx}$$

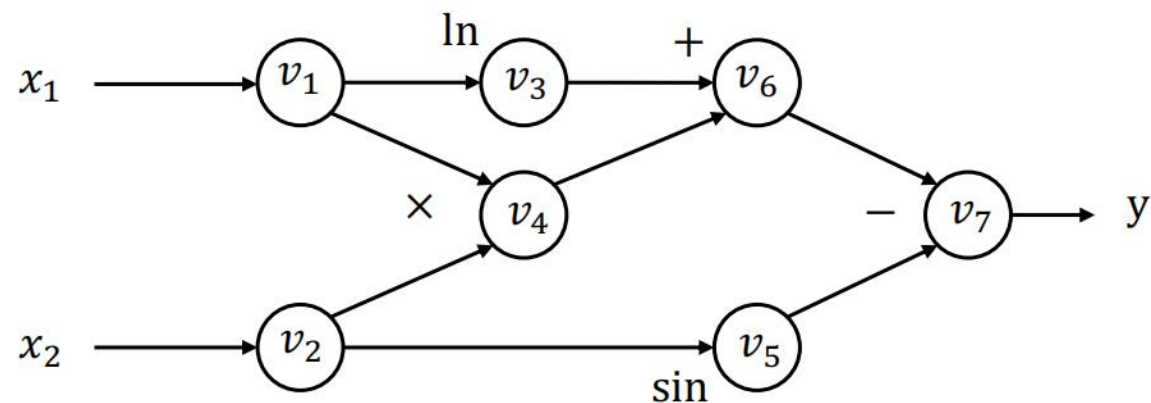
但如果不规划导数计算顺序，可能会导致大量的重复计算，造成计算资源浪费

计算图

计算图是表示复合函数的有向无环图。图中的每个节点都表示一次运算和其运算结果，图中的边表示输入输出关系。比如：

$$y = f(x_1, x_2) = \ln(x_1) + x_1 x_2 - \sin x_2$$

用计算图的形式表示如下：



当 $x_1 = 2, x_2 = 5$ 时，前向运算：

$$v_1 = x_1 = 2$$

$$v_2 = x_2 = 5$$

$$v_3 = \ln v_1 = \ln 2 = 0.693$$

$$v_4 = v_1 v_2 = 10$$

$$v_5 = \sin v_2 = \sin 5 = -0.959$$

$$v_6 = v_3 + v_4 = 10.693$$

$$v_7 = v_6 - v_5 = 11.652$$

$$y = v_7 = 11.652$$

静态计算图

- 定义：静态计算图是一种在开始计算之前就完全定义好的计算图。它在执行前构建，并且图中的每个节点和边都代表一个具体的操作或数据。
- 构建：在静态计算图中，你需要提前定义好所有操作的顺序，包括前向计算和反向传播（用于计算梯度）。
- 优点：
 - 优化：由于计算图是静态的，编译器和硬件可以对其进行优化，提高执行效率。
 - 易于理解：对于简单的模型，静态图易于理解。
- 缺点：
 - 不灵活：对于需要动态决定网络结构的场景，静态图不够灵活。
 - 内存占用：由于整个图需要在内存中构建，对于非常大的图可能会占用大量内存。

静态计算图

```
import tensorflow as tf
```

```
v1 = tf.Variable()
```

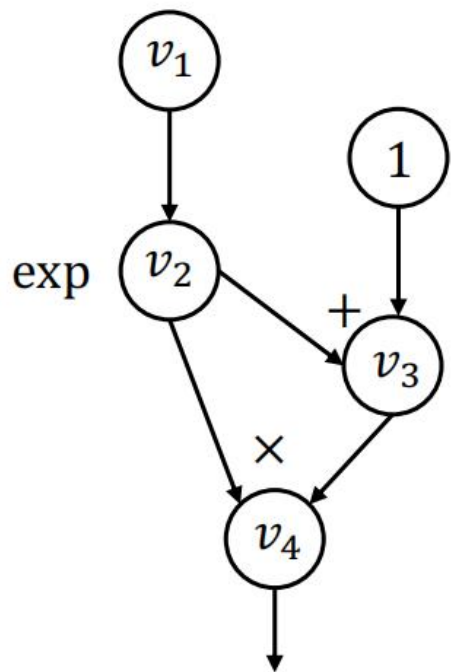
```
v2 = tf.exp(v1)
```

```
v3 = v2 + 1
```

```
v4 = v2 * v3
```

```
sess = tf.Session()
```

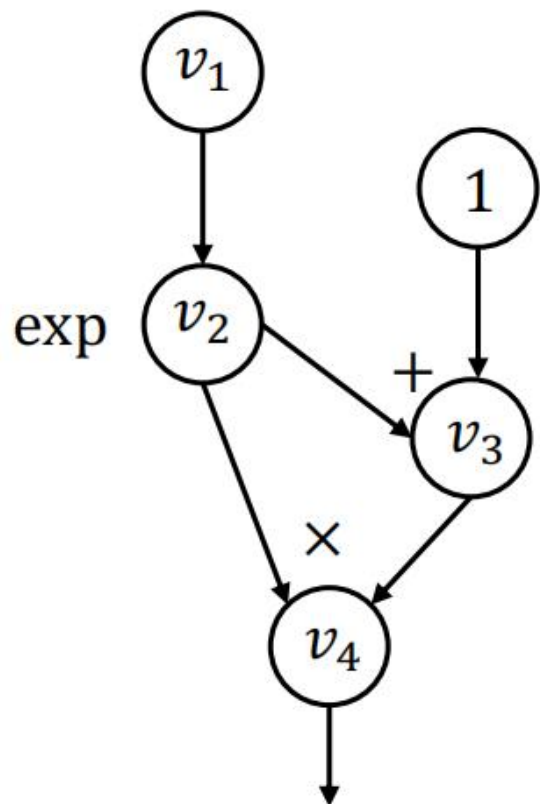
```
value4 = sess.run(v4, feed_dict={v1:  
numpy.array([1])})
```



动态计算图

- 定义：动态计算图是在运行时逐步构建的，可以逐个操作地定义计算图。
- 构建：在动态计算图中，计算图的构建和执行是同步进行的，这意味着你可以在循环中逐个步骤地定义和执行操作。
- 优点：
 - 灵活性：可以轻松处理复杂的控制流和动态图结构。
 - 内存效率：由于计算图是动态构建的，不需要一次性将整个图加载到内存中。
 - Debug：计算图逐步构建与执行，可以在Debug模式下观察变量。
- 缺点：
 - 性能：相比静态图，动态图可能无法充分利用编译器和硬件优化。

动态计算图



```
import torch
```

```
v1 = torch.tensor([1])
```

```
v2 = torch.exp(v1)
```

```
v3 = v2 + 1
```

```
v4 = v2 * v3
```

```
if v4.item() > 0.5:
```

```
    v5 = v4 * 2
```

```
else:
```

```
    v5 = v4
```

```
v5.backward()
```

03

自动微分

前向求导

按拓扑顺序，前向计算每个中间变量对输入变量的导数：

$$v'_i = \frac{\partial v_i}{\partial x_1}$$

$$v'_1 = 1, v'_2 = 0$$

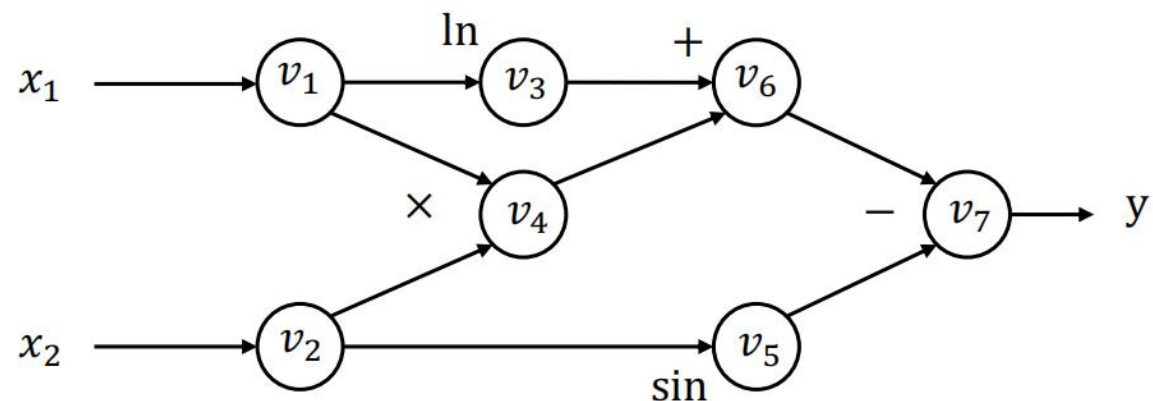
$$v'_3 = \frac{1}{v_1} v'_1 = 0.5$$

$$v'_4 = v'_1 v_2 + v'_2 v_1 = 5$$

$$v'_5 = v'_2 \cos v_2 = 0$$

$$v'_6 = v'_3 + v'_4 = 5.5$$

$$v'_7 = v'_6 - v'_5 = 5.5$$



$$v_1 = x_1 = 2$$

$$v_2 = x_2 = 5$$

$$v_3 = \ln v_1 = \ln 2 = 0.693$$

$$v_4 = v_1 v_2 = 10$$

$$v_5 = \sin v_2 = \sin 5 = -0.959$$

$$v_6 = v_3 + v_4 = 10.693$$

$$v_7 = v_6 - v_5 = 11.652$$

$$y = v_7 = 11.652$$

前向求导

- 对于函数 $f: R^n \rightarrow R^k$ 而言，前向求导需要经过 n 次前向计算，而神经网络中 n 的数量大， k 的数量小
- 为了更有效率地实现自动求导，需要寻找另一种求导方式

反向求导

按逆向拓扑序列，反向计算输出变量对中间变量的导数，即节点 v_i 的**伴随值**(adjoint):

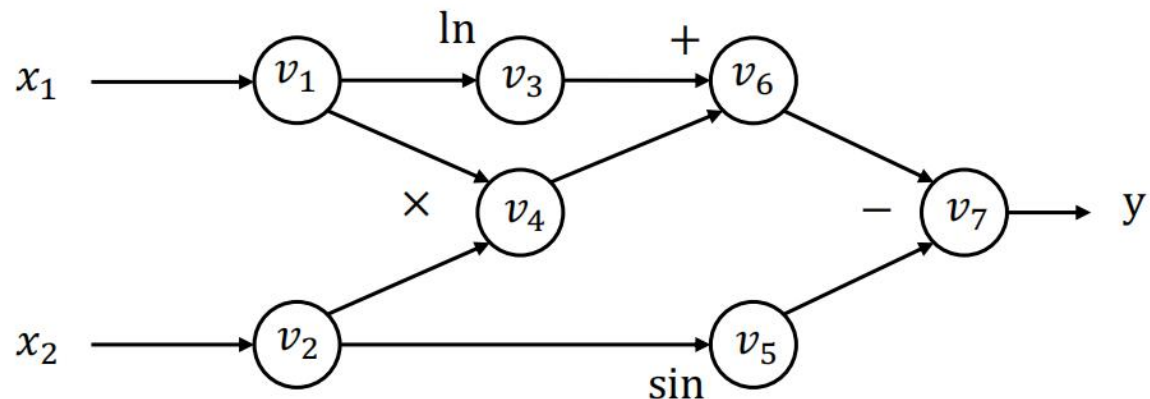
$$\overline{v_i} = \frac{\partial y}{\partial v_i}, \quad \overline{v_7} = 1$$

$$\overline{v_6} = \overline{v_7} \frac{\partial v_7}{\partial v_6} = 1, \quad \overline{v_5} = \overline{v_7} \frac{\partial v_7}{\partial v_5} = -1$$

$$\overline{v_4} = \overline{v_6} \frac{\partial v_6}{\partial v_4} = 1, \quad \overline{v_3} = \overline{v_6} \frac{\partial v_6}{\partial v_3} = 1$$

$$\overline{v_2} = \overline{v_5} \frac{\partial v_5}{\partial v_2} + \overline{v_4} \frac{\partial v_4}{\partial v_2} = \overline{v_5} \cos v_2 + \overline{v_4} v_1 = 1.716$$

$$\overline{v_1} = \overline{v_4} \frac{\partial v_4}{\partial v_1} + \overline{v_3} \frac{\partial v_3}{\partial v_1} = \overline{v_4} v_2 + \overline{v_3} \frac{1}{v_1} = 5.5$$



$$v_1 = x_1 = 2$$

$$v_2 = x_2 = 5$$

$$v_3 = \ln v_1 = \ln 2 = 0.693$$

$$v_4 = v_1 v_2 = 10$$

$$v_5 = \sin v_2 = \sin 5 = -0.959$$

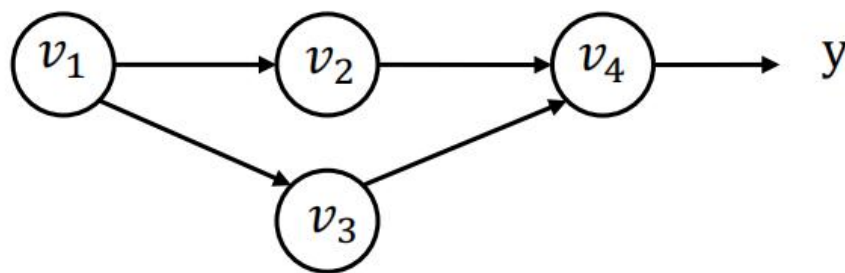
$$v_6 = v_3 + v_4 = 10.693$$

$$v_7 = v_6 - v_5 = 11.652$$

$$y = v_7 = 11.652$$

局部伴随值

需要额外注意的是多输出的节点：



$$\overline{v_1} = \overline{v_2} \frac{\partial v_2}{\partial v_1} + \overline{v_3} \frac{\partial v_3}{\partial v_1}$$

我们定义**局部伴随值**(partial adjoint), 对于计算图中每个连接 $i \rightarrow j$:

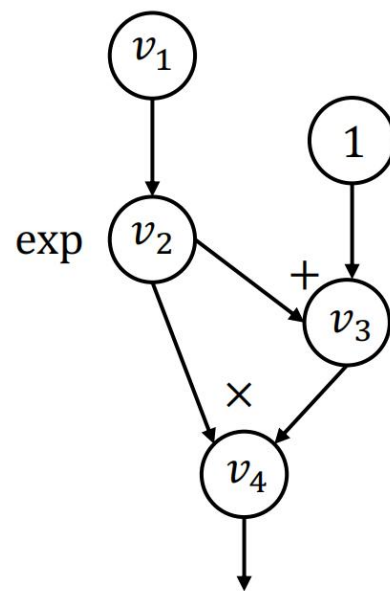
$$\overline{v_{i \rightarrow j}} = \overline{v_j} \frac{\partial v_j}{\partial v_i}, \quad \overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}}$$

计算图的反向求导算法

```
1: def gradient(out):  
2:   node_to_grad = {out: [1]} # 声明一个dict结构, 用于存储partial adjoint  
3:   for i in reverse_topo_order(out): # 从out节点开始进行反向拓扑排序  
4:      $\overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}} = \text{sum}(\text{node\_to\_grad}[i])$  # 求节点i输出边partial adjoint之和  
5:     for k in in(i) :  
6:       compute  $\overline{v_{k \rightarrow i}} = \overline{v_i} \frac{\partial v_i}{\partial v_k}$  # 计算节点i输入边对应的partial adjoint  
7:       append  $\overline{v_{k \rightarrow i}}$  to node_to_grad[k] # 将计算出的partial adjoint存入词典  
8: return adjoint of input  $\overline{v_{input}}$ 
```

反向求导的扩展计算图

```
def gradient(out):  
    node_to_grad = {out: [1]}  
    for i in reverse_topo_order(out):  
         $\overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}} = \text{sum}(\text{node\_to\_grad}[i])$   
        for k in in(i):  
            compute  $\overline{v_{k \rightarrow i}} = \overline{v_i} \frac{\partial v_i}{\partial v_k}$   
            append  $\overline{v_{k \rightarrow i}}$  to node_to_grad[k]  
    return adjoint of input  $\overline{v_{input}}$ 
```

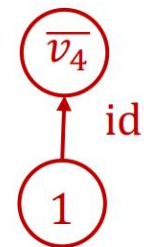
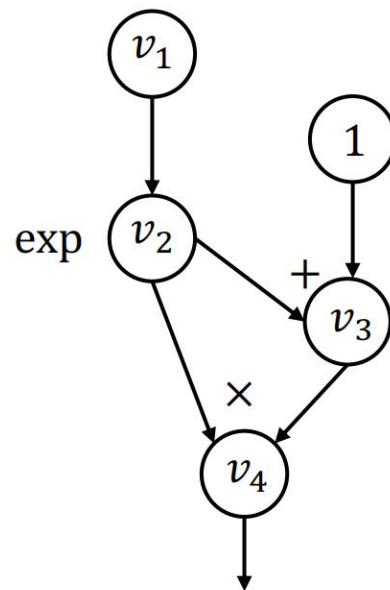


$$y = e^x(e^x + 1)$$

反向求导的扩展计算图

```
def gradient(out):
    node_to_grad = {out: [1]}
    for i in reverse_topo_order(out): # i=4
         $\overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}} = \text{sum}(\text{node\_to\_grad}[i])$ 
        for k in in(i) :
            compute  $\overline{v_{k \rightarrow i}} = \overline{v_i} \frac{\partial v_i}{\partial v_k}$ 
            append  $\overline{v_{k \rightarrow i}}$  to node_to_grad[k]
    return adjoint of input  $\overline{v_{input}}$ 
```

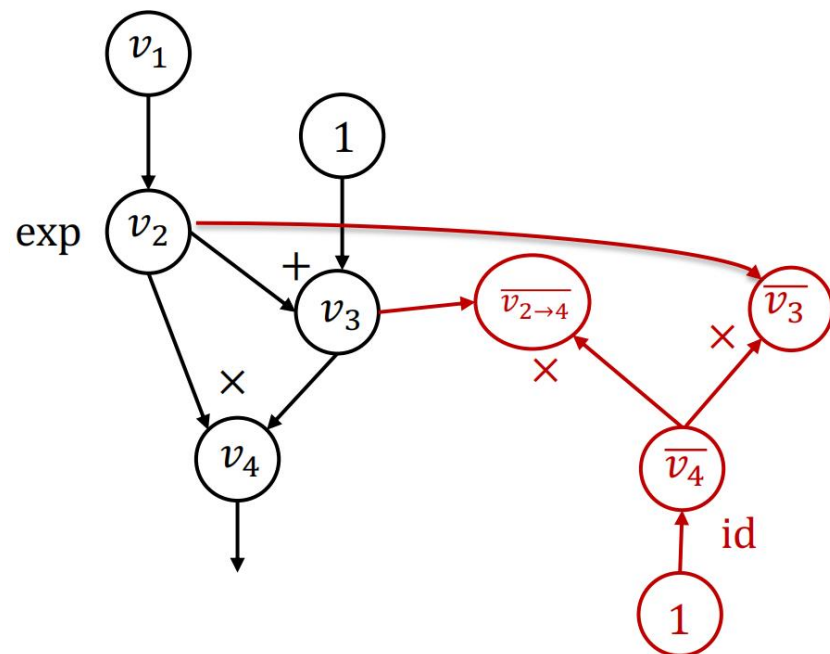
```
node_to_grad:{
    out:[1]
    4:[ $\overline{v_4}$ ]
}
```



反向求导的扩展计算图

```
def gradient(out):
    node_to_grad = {out: [1]}
    for i in reverse_topo_order(out): # i=4
         $\overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}} = \text{sum}(\text{node\_to\_grad}[i])$ 
        for k  $\in$  in(i) :
            compute  $\overline{v_{k \rightarrow i}} = \overline{v_i} \frac{\partial v_i}{\partial v_k}$ 
            append  $\overline{v_{k \rightarrow i}}$  to node_to_grad[k]
    return adjoint of input  $\overline{v_{input}}$ 
```

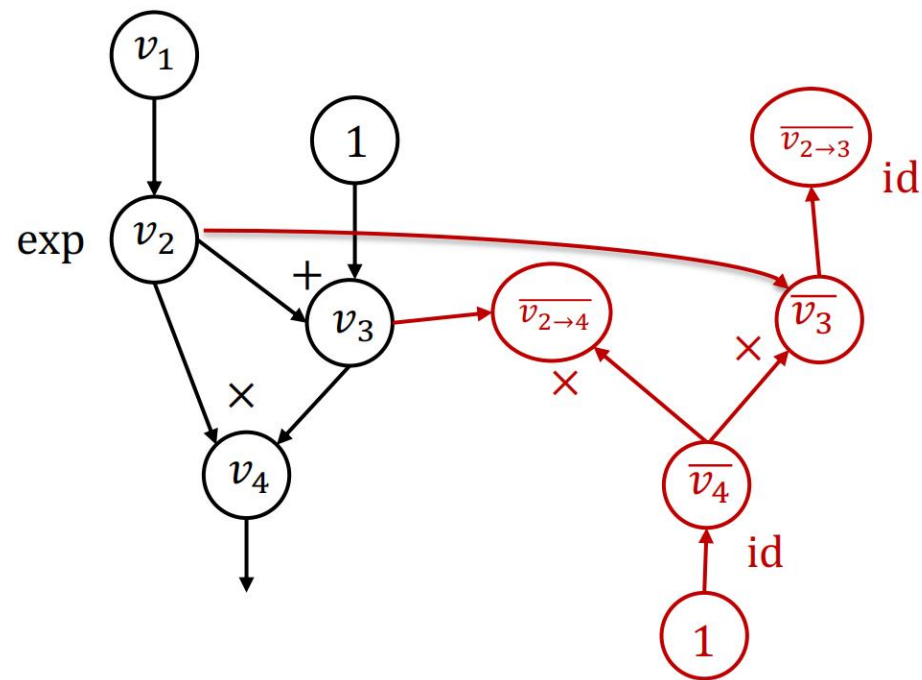
```
node_to_grad:{
    out:[1]
    4:[ $\overline{v_4}$ ]
    3:[ $\overline{v_3}$ ]
    2:[ $\overline{v_{2 \rightarrow 4}}$ ]
}
```



反向求导的扩展计算图

```
def gradient(out):
    node_to_grad = {out: [1]}
    for i in reverse_topo_order(out): # i=3
         $\overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}} = \text{sum}(\text{node\_to\_grad}[i])$ 
        for k in in(i):
            compute  $\overline{v_{k \rightarrow i}} = \overline{v_i} \frac{\partial v_i}{\partial v_k}$ 
            append  $\overline{v_{k \rightarrow i}}$  to node_to_grad[k]
    return adjoint of input  $\overline{v_{input}}$ 
```

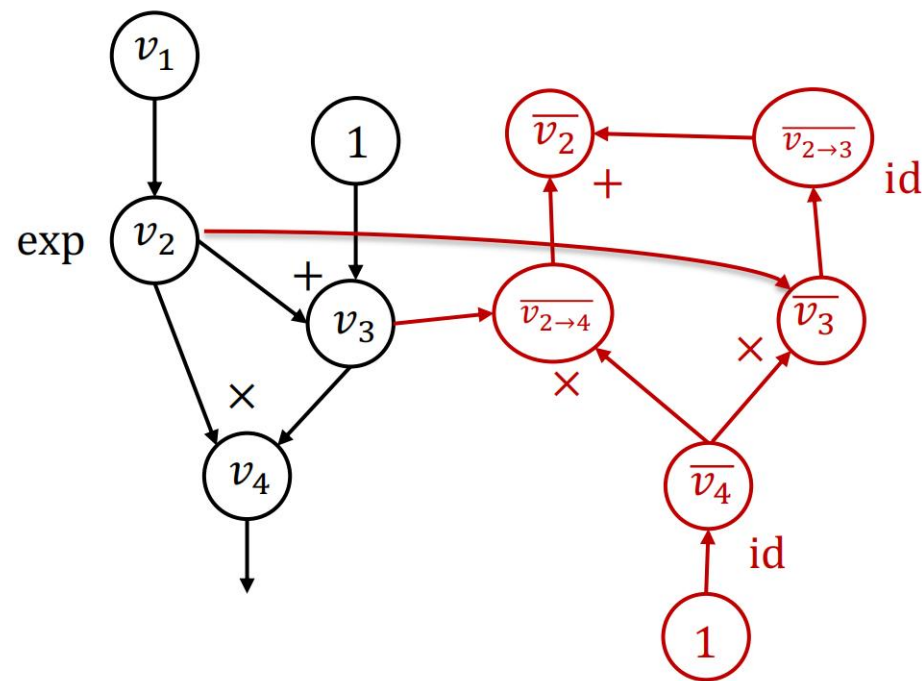
```
node_to_grad:{
    out:[1]
    4:[ $\overline{v_4}$ ]
    3:[ $\overline{v_3}$ ]
    2:[ $\overline{v_{2 \rightarrow 4}}$ ,  $\overline{v_{2 \rightarrow 3}}$ ]
}
```



反向求导的扩展计算图

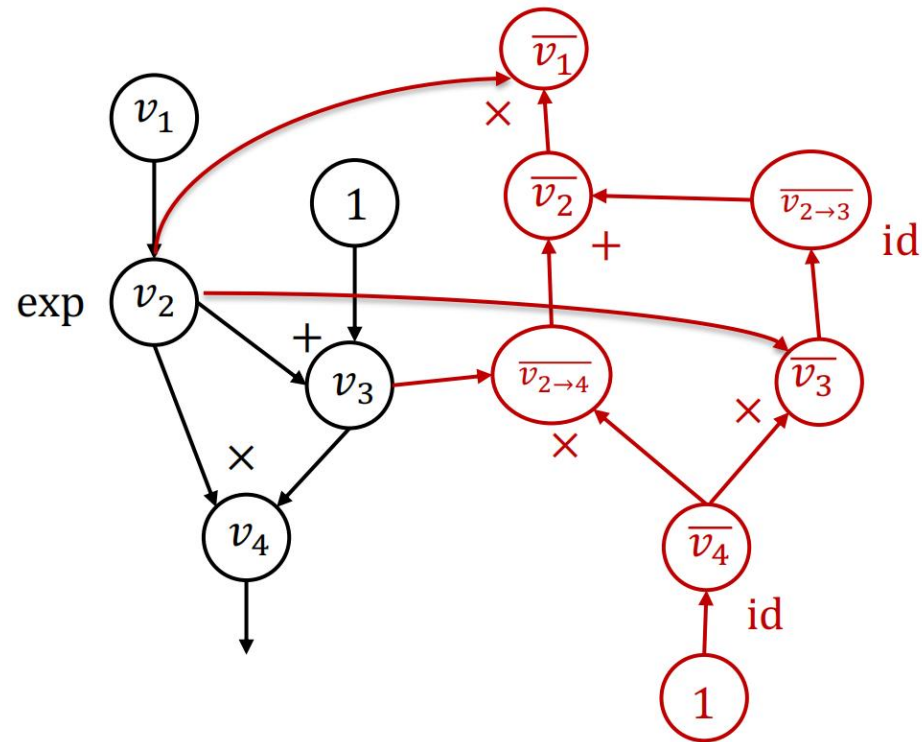
```
def gradient(out):
    node_to_grad = {out: [1]}
    for i in reverse_topo_order(out): # i=2
         $\overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}} = \text{sum}(\text{node\_to\_grad}[i])$ 
        for k in in(i):
            compute  $\overline{v_{k \rightarrow i}} = \overline{v_i} \frac{\partial v_i}{\partial v_k}$ 
            append  $\overline{v_{k \rightarrow i}}$  to node_to_grad[k]
    return adjoint of input  $\overline{v_{input}}$ 
```

```
node_to_grad:{
    out:[1]
    4:[ $\overline{v_4}$ ]
    3:[ $\overline{v_3}$ ]
    2:[ $\overline{v_{2 \rightarrow 4}}$ ,  $\overline{v_{2 \rightarrow 3}}$ ]
}
```



反向求导的扩展计算图

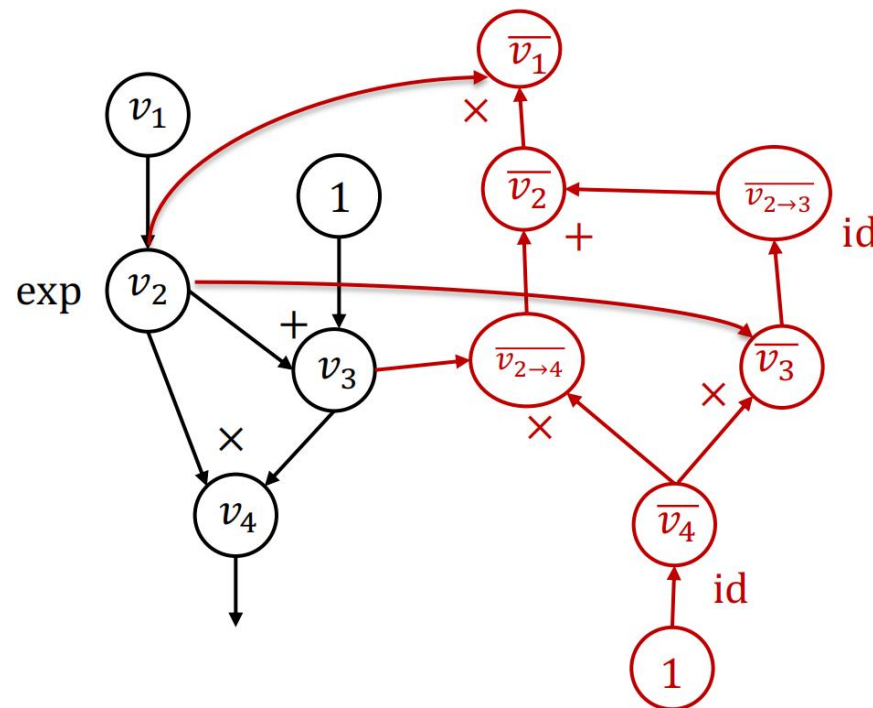
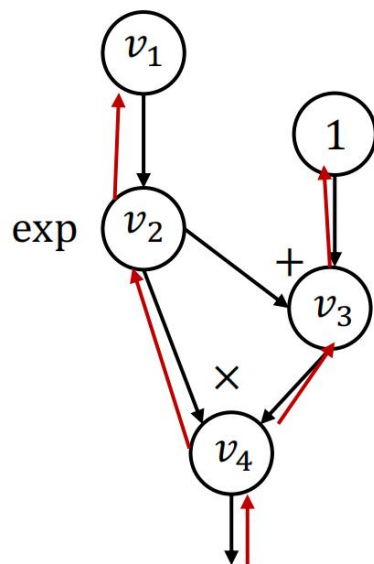
```
def gradient(out):
    node_to_grad = {out: [1]}
    for i in reverse_topo_order(out): # i=2
         $\overline{v_i} = \sum_{j \in out(i)} \overline{v_{i \rightarrow j}} = \text{sum}(\text{node\_to\_grad}[i])$ 
        for k in in(i):
            compute  $\overline{v_{k \rightarrow i}} = \overline{v_i} \frac{\partial v_i}{\partial v_k}$ 
            append  $\overline{v_{k \rightarrow i}}$  to node_to_grad[k]
    return adjoint of input  $\overline{v_{input}}$ 
node_to_grad: {
    out: [1]
    4: [ $\overline{v_4}$ ]
    3: [ $\overline{v_3}$ ]
    2: [ $\overline{v_{2 \rightarrow 4}}$ ,  $\overline{v_{2 \rightarrow 3}}$ ]
    1: [ $\overline{v_1}$ ]
}
```



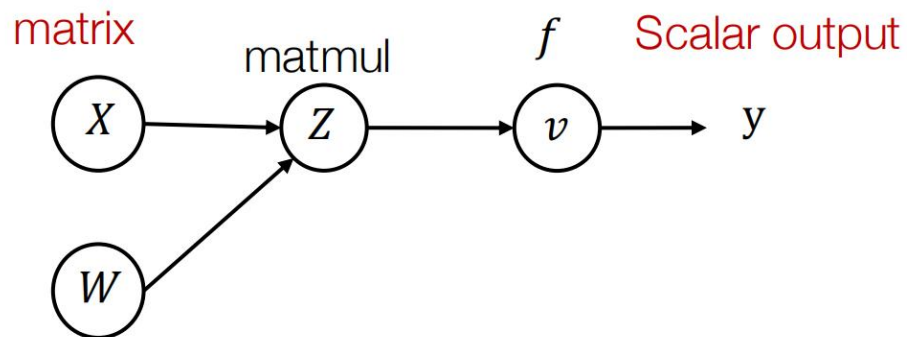
反向求导的扩展计算图

反向求导能够被建模为扩展计算图，也属于计算图的一种，可以用来进一步计算梯度的梯度。

早期框架（caffe等）采用反向传播，而现代框架（PyTorch, TensorFlow）主要使用扩展计算图。



张量的反向求导



$$Z_{ij} = \sum_k X_{ik} W_{kj}, \quad Z = XW$$

$$v = f(Z)$$

节点Z的伴随值:

$$\bar{Z} = \begin{pmatrix} \frac{\partial y}{\partial Z_{11}} & \cdots & \frac{\partial y}{\partial Z_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial y}{\partial Z_{m1}} & \cdots & \frac{\partial y}{\partial Z_{mn}} \end{pmatrix}$$

$$\bar{X}_{ik} = \sum_j \frac{\partial Z_{ij}}{\partial X_{ik}} \bar{Z}_{ij} = \sum_j W_{kj} \bar{Z}_{ij}$$

$$\bar{X} = \bar{Z}W$$

代码实现：计算图节点类

```
class Value:
```

```
    op: Optional[Op] # 节点对应的计算操作, Op是自定义的计算操作类
```

```
    inputs: List["Value"]
```

```
    cached_data: NDArray
```

```
    requires_grad: bool
```

```
    def realize_cached_data(self): # 进行计算得到节点对应的变量, 存储在cached_data里
```

```
    def is_leaf(self):
```

```
    def _init(self, op: Optional[Op], inputs: List["Value"], *, num_outputs: int = 1,  
cached_data: NDArray = None, requires_grad: Optional[bool] = None):
```

```
    @classmethod
```

```
    def make_const(cls, data, *, requires_grad=False): # 建立一个用data生成的独立节点
```

```
    def make_from_op(cls, op: Op, inputs: List["Value"]): # 用op和inputs计算, 生成节点
```

代码实现：计算操作类

```
class Op(ABC):
```

```
    @abstractmethod
```

```
    def compute(self, *args: Tuple["NDArray"]) -> NDArray:
```

```
        # 前向计算. 参数args是由NDArray组成的序列Tuple, 输出计算的结果NDArray
```

```
        pass
```

```
    @abstractmethod
```

```
    def gradient(self, out_grad: "Value", node: "Value") -> Tuple["Value"]:
```

```
        # 后向求导. 计算每个输入变量对应的局部伴随值(partial adjoint)
```

```
        # 参数out_grad是输出变量对应的伴随值, node是计算操作所在的计算图节点
```

```
        # 为方便编程, 输出总是一个序列Tuple
```

```
        pass
```

代码实现：节点类成员函数

```
def Value.realize_cached_data(self):  
    if self.is_leaf() or self.cached_data is not None:  
        return self.cached_data  
  
    self.cached_data = self.op.compute(  
        *[x.realize_cached_data() for x in self.inputs]  
    )  
    return self.cached_data  
  
def Value.is_leaf(self):  
    return self.op is None
```

代码实现：张量类

```
class Tensor (Value):
    grad: "Tensor"
    def __init__(self, array, *, dtype=None,
requires_grad=True, **kwargs):

    @staticmethod
    def from_numpy(numpy_array, dtype):
    @staticmethod
    def make_from_op(op: Op, inputs: List["Value"]):
    @staticmethod
    def make_const(data, requires_grad=False):
```

```
@ property
def data (self): #封装Value.cached_data
@ data.setter
def data (self, value):
@ property
def shape (self):
@ property
def dtype (self):

def backward (self, out_grad=None):
def detach (self):
def __add__ (self, other):
def __sub__ (self, other):
.....
```


代码实现：张量类

```
def detach(self):  
    # 创建一个新的张量，但不接入计算图  
    return Tensor.make_const(self.realize_cached_data())  
  
@staticmethod  
def make_const(data, requires_grad=False):  
    tensor = Tensor.__new__(Tensor)  
    if isinstance(data, Tensor)  
        tensor_data = data.realize_cached_data()  
    else  
        tensor_data = data  
    tensor._init(None, [], # 将前置节点置空  
        cached_data = tensor_data, requires_grad = requires_grad)  
    return tensor
```

代码实现：张量类

```
@staticmethod
```

```
def make_from_op(op: Op, inputs: List["Value"]):
```

```
    tensor = Tensor.__new__(Tensor)
```

```
    tensor._init(op, inputs)
```

```
    if not LAZY_MODE: #LAZY_MODE用于实现先建图后计算 （静态计算图，作业中无需实现）
```

```
        tensor.realize_cached_data()
```

```
    return tensor
```

```
class TensorOp(Op):
```

```
    # 继承计算操作类，实现张量特有的计算
```

```
    def __call__(self, *args):
```

```
        return Tensor.make_from_op(self, args)
```

代码实现：张量计算

<code>class EwiseAdd(TensorOp):</code>	<code># 对应元素相加</code>
<code>class AddScalar(TensorOp):</code>	<code># 加常数</code>
<code>class EwiseMul(TensorOp):</code>	<code># 对应元素乘</code>
<code>class MulScalar(TensorOp):</code>	<code># 乘常数</code>
<code>class PowerScalar(TensorOp):</code>	<code># 常数幂</code>
<code>class EwiseDiv(TensorOp):</code>	<code># 对应元素除</code>
<code>class DivScalar(TensorOp):</code>	<code># 除以常数</code>
<code>class Transpose(TensorOp):</code>	<code># 矩阵转置</code>
<code>class Reshape(TensorOp):</code>	<code># 变形</code>
<code>class BroadcastTo(TensorOp):</code>	<code># 广播</code>
<code>class Summation(TensorOp):</code>	<code># 按维度求和</code>
<code>class MatMul(TensorOp):</code>	<code># 矩阵相乘</code>
<code>class Negate(TensorOp):</code>	<code># 求相反数</code>
<code>class Log(TensorOp):</code>	<code># 求对数</code>
<code>class Exp(TensorOp):</code>	<code># 求指数</code>

代码实现：张量计算

```
class EwiseAdd(TensorOp):
    def compute(self, a: NDArray, b: NDArray):
        return a + b

    def gradient(self, out_grad: Tensor, node: Tensor):
        return out_grad, out_grad

class AddScalar(TensorOp):
    def __init__(self, scalar):
        self.scalar = scalar

    def compute(self, a: NDArray):
        return a + self.scalar

    def gradient(self, out_grad: Tensor, node: Tensor):
        return (out_grad,) #重载gradient函数的输出必须是Tuple
```

```
def Tensor.__add__(self, other):
    if isinstance(other, Tensor):
        return EwiseAdd()(self, other)
    else:
        return AddScalar(other)(self)
```



实现张量计算之后，在
Tensor类中进行计算符重载

代码实现：自动求导

```
def Tensor.backward(self, out_grad=None):  
    # 最后一个节点时, out_grad为1  
    if out_grad is not None:  
        out_grad = out_grad  
    else:  
        out_grad = Tensor(numpy.ones(self.shape))  
    compute_gradient_of_variables(self, out_grad)
```

代码实现：自动求导

```
def compute_gradient_of_variables(output_tensor, out_grad):  
    node_to_output_grads_list: Dict[Tensor, List[Tensor]] = {} # dict结构, 用于存储partial adjoint  
    node_to_output_grads_list[output_tensor] = [out_grad]  
    reverse_topo_order = list(reversed(topo_sort([output_tensor]))) # 请自行实现拓扑排序函数  
  
    for node in reverse_topo_order:  
        node.grad = sum(node_to_output_grads_list[node]) # 求node的partial adjoint之和, 存入属性grad  
        if node.is_leaf():  
            continue  
        for i, grad in enumerate(node.op.gradient(node.grad, node)): # 计算node.inputs的partial adjoint  
            j = node.inputs[i]  
            if j not in node_to_output_grads_list:  
                node_to_output_grads_list[j] = []  
            node_to_output_grads_list[j].append(grad) # 将计算出的partial adjoint存入dict
```

代码实现：自动求导

- 计算图在执行TensorOP的__call__函数时被隐式创建

以张量加法为例，调用链是：

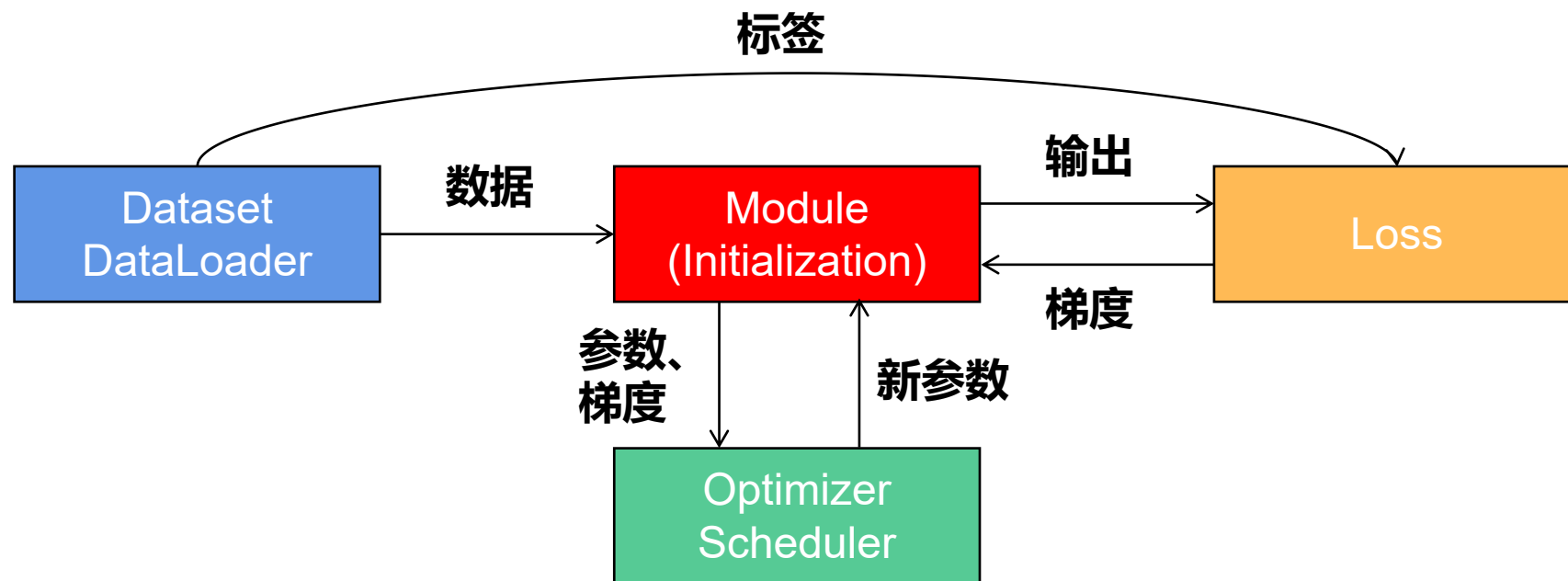
$c=a+b$ ➞ `Tensor.__add__()` ➞ `EwiseAdd.__call__()` ➞ `Tensor.make_from_op()`
➞ `Op.compute()`

- 每个节点只知道其输入节点，不知道输出节点
- 代码编写完成后，需结合数值微分方法进行测试

04

神经网络框架实现

神经网络编程框架：模块化



神经网络编程框架：模块化

Dataset & Data loader: 加载数据集、划分mini-batch、数据预处理

Module: 神经网络模块化结构

Initialization: 初始化网络参数

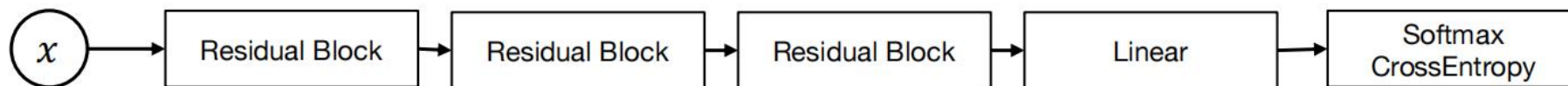
Loss: 损失函数

Optimizer: 接收 `nn.Module` 传入的模型参数与梯度，进行参数更新

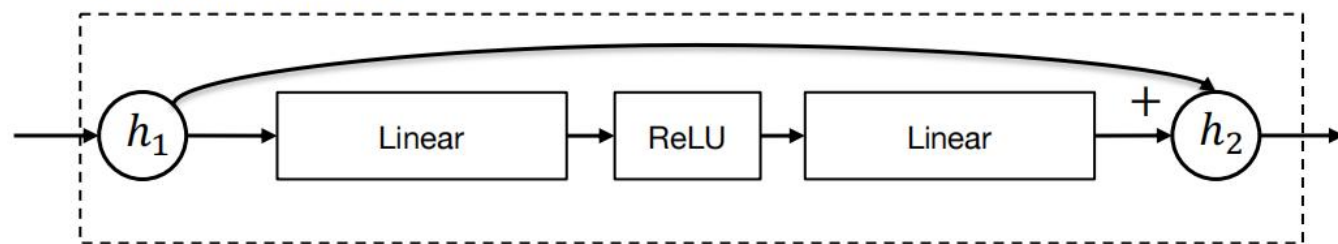
Scheduler: 规划学习率

神经网络编程框架：模块化

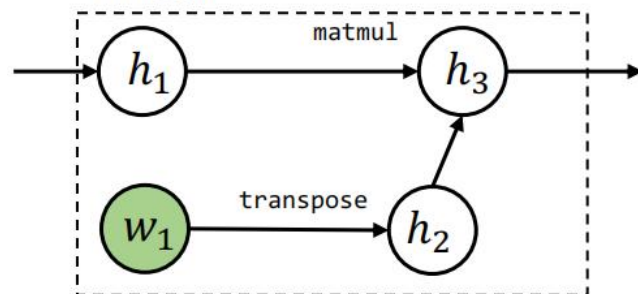
Multi-layer Residual Net



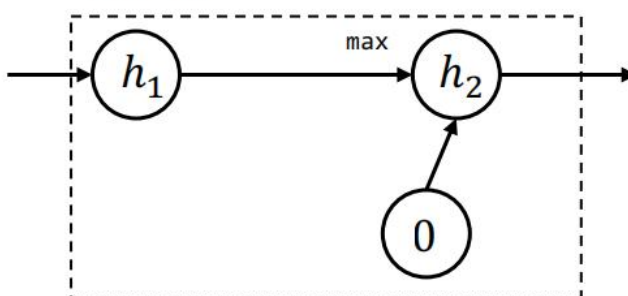
Residual block



Linear



ReLU



代码实现：神经网络模块

```
class Module(ABC):
    def __init__(self):
        self.training = True

    def parameters(self) -> List["Tensor"]:
        return _unpack_params(self.__dict__)

    def _children(self) -> List["Module"]:
        return _child_modules(self.__dict__)

    def __call__(self, *args, **kwargs):
        return self.forward(*args, **kwargs)

    @abstractmethod
    def forward(self):
        pass
```

```
def eval(self):
    self.training = False
    for m in self._children():
        m.training = False

def train(self):
    self.training = True
    for m in self._children():
        m.training = True
```

代码实现：神经网络模块

```
class Parameter(Tensor): # 声明一个类专门表示网络参数

def _unpack_params(value: object) -> List[Tensor]:
    if isinstance(value, Parameter): return [value]
    elif isinstance(value, Module): return value.parameters()
    elif isinstance(value, dict):
        params = []
        for k, v in value.items():
            params += _unpack_params(v)
        return params
    elif isinstance(value, (list, tuple)):
        params = []
        for v in value:
            params += _unpack_params(v)
        return params
    else: return []
```

代码实现：神经网络模块

```
def _child_modules(value: object) -> List["Module"]:  
    if isinstance(value, Module):  
        modules = [value]  
        modules.extend(_child_modules(value.__dict__))  
        return modules  
    if isinstance(value, dict):  
        modules = []  
        for k, v in value.items():  
            modules += _child_modules(v)  
        return modules  
    elif isinstance(value, (list, tuple)):  
        modules = []  
        for v in value:  
            modules += _child_modules(v)  
        return modules  
    else:  
        return []
```

代码实现：神经网络模块

<code>class Linear(Module)</code>	<code># 线性层</code>
<code>class Flatten(Module)</code>	<code># 平铺层</code>
<code>class ReLU(Module)</code>	<code># ReLU激活函数</code>
<code>class Sigmoid(Module)</code>	<code># Sigmoid激活函数</code>
<code>class Softmax(Module)</code>	<code># Softmax层</code>
<code>class CrossEntropyLoss(Module)</code>	<code># 交叉熵损失</code>
<code>class BinaryCrossEntropyLoss(Module)</code>	<code># 二元交叉熵损失</code>
<code>class MSELoss(Module)</code>	<code># 均方损失</code>
<code>class BatchNorm1d(Module)</code>	<code># 一维批归一化 (选做)</code>
<code>class LayerNorm1d(Module)</code>	<code># 一维层归一化 (选做)</code>
<code>class Dropout(Module)</code>	<code># Dropout层 (选做)</code>
<code>class Sequential(Module)</code>	<code># 多层模型</code>
<code>class Residual(Module)</code>	<code># 残差连接</code>

代码实现：神经网络模块

```
class Linear(Module):
    def __init__(self, in_features, out_features, bias=True, dtype="float32"):
        super().__init__()
        self.in_features = in_features
        self.out_features = out_features
        self.weight = init_He(in_features, out_features, dtype) #请自行实现初始化算法
        if bias:
            self.bias = Tensor(numpy.zeros(self.out_features))
        else:
            self.bias = None

    def forward(self, X: Tensor) -> Tensor:
        X_out = X @ self.weight
        if self.bias:
            return X_out + self.bias.broadcast_to(X_out.shape)
        return X_out
```


代码实现：神经网络模块

```
class Sequential(Module):  
    def __init__(self, *modules):  
        super().__init__()  
        self.modules = modules  
  
    def forward(self, x: Tensor) -> Tensor:  
        for module in self.modules:  
            x = module(x)  
        return x
```

代码实现： 优化器与学习率规划器

```
class SGD(Optimizer)                # 基本随机梯度下降
class SGD_WeightDecay(Optimizer)    # L2 (L1) 正则化
class Momentum(Optimizer)           # 动量法 (Nestrov) , 以上三项可合并
class Adam(Optimizer)               # Adam
class StepDecay(Scheduler)          # 阶梯型衰减
class LinearWarmUp(Scheduler)       # 线性热启动
class CosineDecayWithWarmRestarts(Scheduler) # 带热重启的Cosine衰减
```

代码实现： 优化器

```
class Optimizer(ABC):  
    def __init__(self, params):  
        self.params = params  
  
    @abstractmethod  
    def step(self):  
        pass  
  
    def reset_grad(self):  
        for p in self.params:  
            p.grad = None
```

代码实现： 优化器

```
class SGD(Optimizer):  
    def __init__(self, params, lr = 0.001):  
        super().__init__(params)  
        self.lr = lr  
  
    def step(self):  
        for i, param in enumerate(self.params):  
            grad = Tensor(param.grad, dtype='float32').data  
            param.data = param.data - grad * self.lr
```

作业要求

1. 实现DataSet、DataLoader类，能够从表格数据文件、图像文件读取Iris、MNIST、Fashion-MNIST数据集；
2. 实现ppt中列举的Module、Optimizer、Scheduler子类，实现He和Xavier初始化算法；实现模块化的神经网络架构；
3. 使用简易框架进行ResidualMLP实验，实现模型存储和读取，测试阶段不计算梯度；
4. 实现Value、Op、Tensor等类，实现自动求导并测试，实现ppt中列举的TensorOp子类 (选做)。

DDL：2026-7-10，超星教学平台提交，请留意作业通知。

鼓励：

创新思路，面向对象编程与设计模式